

1. Explain the difference between generative and discriminative models. Give one example of each.

Discriminative models: learn to predict label y from data x , modeled as $P(Y | X)$.

Don't care how data was generated. Example : Logistic Regression

Generative models: learn to predict label y from data x , and know how the data was generated, modeled as $P(X | Y)$ and $P(Y)$. Example: GAN

2. What is the purpose of transformers in modern AI? How does it work briefly?

They solved a key limitation of older architectures (RNNs/LSTMs): the inability to handle long-range dependencies efficiently and the inability to parallelize training. The core idea is the Attention Mechanism — instead of processing words one at a time, a transformer looks at the *entire sequence at once* and asks: *"which other words should I pay attention to when understanding this word?"*

3. Describe the difference between supervised, unsupervised, and reinforcement learning

Supervised: The model is trained on labeled data

Unsupervised: The model is trained on unlabeled data

Reinforcement learning: The model tries if he is true, he gets trait. if he is wrong, he gets punished

4. What are tokens in the context of Large Language Models? Why are they important?

A token is the basic unit of text that an LLM processes.

Tokens matter for several reasons Context window, Unit of processing and Cost & speed

5. Explain the concept of attention in neural networks. How does it improve model performance?

Attention is a mechanism that allows a model to focus on the most relevant parts of the input when producing an output, rather than compressing everything into a single vector. For each token, it computes a relevance score against all other tokens using Query, Key, and Value vectors, then takes a weighted sum — giving more weight to important tokens.

It improves performance by capturing long-range dependencies, enabling parallel processing, and making the same word mean different things in different contexts.

6. What is prompt engineering, and why is it crucial for Gen AI applications? State the different types of prompts used.

Prompt engineering is the practice of carefully designing inputs (prompts) to guide an LLM toward producing accurate, relevant, and useful outputs.
Few-Shot, Zero-Shot, Template-based.

7. Explain how RAG (Retrieval-Augmented Generation) works.

1. Indexing (Offline)

- External documents (PDFs, websites, databases) are chunked and converted into vector embeddings and stored in a vector database (e.g., Pinecone, FAISS).

2. Retrieval (At Query Time)

- The user's query is also converted into an embedding and used to search the vector database for the most semantically similar chunks.

3. Augmentation

- The retrieved chunks are injected into the prompt alongside the original query as context.

4. Generation

- The LLM reads both the query and retrieved context to generate a grounded, accurate response.

8. What is embedding in AI, and why is it important for retrieval tasks?

An embedding is a numerical vector representation of data (text, images, audio) that captures its semantic meaning in a high-dimensional space. Similar meanings are placed closer together in this space, allowing machines to measure relationships mathematically.

9. How does temperature affect the output of a language generation model?

Temperature is a parameter that controls the randomness and creativity of a model's output by adjusting the probability distribution over the next token before sampling.

10. What is chain-of-thought prompting, and why can it improve reasoning in LLMs?

Instructs the model to reason step by step.

As it gives the model the way of thinking and steps to solve the problem

11. What is the difference between RAG and fine-tuning? Give a use case for each.

The RAG no need to retrain the model you need to convert the data into embeddings

On the other hand fine-tuning needs to retrain the model which takes more time and power but will be faster in execution.

If u have some rules or book of instruction it will be better to use RAG as the model will only go and search about what the user ask . on the other hand if you have medical model and you need it to diagnose certain diseases it will be better to use fine-tuning as it will be more accurate.

12. Explain the possible improvements that can be applied to a Naive RAG (Retrieval-Augmented Generation) system, both pre-retrieval and post-retrieval.

Pre-Retrieval improvements focus on bettering the query and index before searching — techniques like query rewriting, query expansion, HyDE, and semantic chunking ensure the right documents are found. Post-Retrieval improvements refine what was retrieved before passing it to the LLM — techniques like re-ranking, redundancy removal, and context compression ensure only the most relevant, clean context is used. Together, these improvements fix the two core weaknesses of Naive RAG: *finding the wrong chunks* and *sending noisy context to the model*.

13. What factors should you consider when choosing an LLM for an application?

Consider accuracy, latency, cost, context window, privacy, and reasoning ability.

When choosing an LLM, consider accuracy and reasoning ability for complex tasks, latency and cost for high-traffic or real-time applications, and context window size for document-heavy pipelines like RAG. Privacy is critical when handling sensitive data — requiring on-premise or private deployment over public

APIs. The right LLM is never universal — it's always a tradeoff based on your application's priorities.

14. How can you evaluate whether an LLM-generated answer is grounded in the provided documents?

Groundedness can be evaluated by checking if every claim in the generated answer can be traced back to a retrieved document using methods like faithfulness scoring, citation verification, or LLM-as-Judge.

15. How do you think computer vision could be integrated with Generative AI ?

Computer vision gives Gen AI eyes — enabling models to analyze, understand, and generate visual content like images, videos, and documents, powering applications from medical diagnosis to autonomous agents.

16.

a. System architecture

Here's how I'd lay it out, with the reasoning behind each piece.

Frontend / access layer. Team members log in through a single chat interface (web or Slack/Teams plugin). This layer handles authentication and passes the user's identity and team membership down to every other component — it's the anchor for the access control requirement later.

Identity and access management (IAM). Sits behind login, tied into CURT's existing SSO (Azure AD, Okta, whatever they use). Every request carries the user's team (Mechanical, Electrical, Software, Business) and role. This is what lets the retrieval layer filter results before anything reaches the model.

Query orchestrator / router. Receives the question, decides whether it needs unstructured retrieval (RAG), structured data (a database query), or both, and coordinates the rest of the pipeline. For a question like "what's our current battery cell supplier cost," it would route to structured data; for "what's the failure mode we found in the Q2 suspension tests," it would route to RAG.

Ingestion pipeline. Pulls in documents from wherever each team stores them (SharePoint, Drive, Confluence, local drives, CAD repositories), converts them to text, cleans them, chunks them, and generates embeddings. Runs continuously or on a schedule so new documents show up in the index without a manual push.

Vector store. Holds the embeddings plus metadata (source file, team, author, date, access level). Metadata is what makes both citation and access control possible — retrieval can filter by team before results ever reach the LLM.

Structured data connector. A read-only interface (SQL wrapper, API layer, or a text-to-SQL tool) into CURT's databases — test logs, part inventories, budgets, sensor data. Keeps structured lookups separate from document retrieval since they need different handling.

LLM / generation layer. Takes the retrieved chunks or structured query results plus the user's question, and generates the answer. Prompted to only answer from what it was given and to attach source references to every claim.

Citation and source tracker. Keeps the link between each generated sentence and the chunk or record it came from, so the final answer can show "Source: Suspension_Test_Report_Q2.pdf, page 4" rather than an unsupported claim.

Access control / guardrail layer. Checks, at both retrieval and generation time, that the user is only seeing content their team is cleared for. Business team members shouldn't be able to pull proprietary electrical schematics even if they ask directly.

Evaluation and logging module. Records every query, retrieved sources, and generated answer, and periodically scores them (automated + human review) for accuracy, relevance, and citation correctness.

A rough flow: user asks a question → IAM tags it with team/role → orchestrator decides RAG vs. structured query vs. both → retrieval (filtered by access) or database call happens → LLM generates an answer grounded in what came back, with citations attached → guardrail layer double-checks the answer doesn't leak restricted content → answer returned to the user → the exchange gets logged for evaluation.

b. Main challenges

Data preprocessing and file formats. CURT's documents will span PDFs, Word files, Excel sheets, CAD drawings, scanned lab notes, maybe even images of whiteboards. Each format needs a different extraction method, and information gets lost easily — tables collapse into unreadable text, CAD metadata doesn't map to plain language, scanned pages need OCR before anything else can happen.

Inconsistent, incomplete, or noisy data. Engineering docs get versioned informally ("draft_v3_FINAL_actual.docx"), test logs have missing fields, and teams don't always follow the same naming or formatting conventions. Old documents may directly contradict newer ones with no indication which is current.

Chunking and context limits. A 40-page test report can't go into the model whole. Chunking has to preserve enough context that a chunk still makes sense on its own — cut it wrong and you get a paragraph referencing "the table above" with no table attached.

Retrieval relevance. Team-specific jargon and acronyms (CURT almost certainly has plenty) can cause semantic search to miss the right chunk or return one that's topically close but actually wrong. Cross-team questions — say, a mechanical failure traced to an electrical issue — make this harder, since the answer might live in a different team's document set entirely.

Answer accuracy. Even with good retrieval, the model can still blend information from multiple sources incorrectly, state something confidently that isn't supported by the retrieved text, or miss a caveat buried in the source document (e.g., "valid only under load < 200N").

c. Techniques to address these

Format-aware ingestion pipelines. Use dedicated parsers per file type — pdfplumber or OCR (Tesseract, or a vision model) for PDFs and scans, python-docx for Word, openpyxl or a table-specific extractor for Excel — rather than one generic text extractor. Preserve table structure (as markdown tables or structured JSON) instead of flattening it to plain text, which is where a lot of the useful information in engineering docs actually lives.

Metadata tagging and versioning at ingestion. Every chunk gets tagged with source file, team, date, and version at ingestion time. A simple rule (most recent version wins, or superseded docs get flagged) keeps outdated information from competing with current information in retrieval results, and gives the model something to point to when a user asks which version is current.

Semantic + hybrid chunking. Instead of fixed-size splitting, chunk along natural document boundaries (sections, paragraphs, table units) with some overlap between chunks so context isn't cut at an arbitrary point. For technical documents, keep tables and their surrounding explanation together as one chunk.

Hybrid retrieval (dense + sparse). Combine vector similarity search with keyword/BM25 search so team-specific acronyms and part numbers — the kind of terms embeddings sometimes miss — still surface correctly. A reranking step after initial retrieval helps push the most relevant chunks to the top before they reach the model.

Grounded generation with citation enforcement. Prompt the model to answer only from the retrieved context and to cite the specific chunk for each claim; if nothing relevant was retrieved, it should say so instead of guessing. This is also where the

structured-data path helps — for numeric or record-based questions, pulling the exact value from a database is more reliable than asking the model to reason over document text.

Confidence scoring and fallback. If retrieval confidence is low (weak similarity scores, no strong match), the assistant should say it isn't sure rather than answer anyway, and can offer to route the question to a person on the relevant team.

d. Evaluating accuracy and reliability

Retrieval evaluation. Build a test set of representative questions per team, each with a known correct source document. Measure precision and recall — did the system retrieve the right chunk, and did it avoid pulling in irrelevant ones.

Answer evaluation. For the same test set, score generated answers against reference answers using a mix of automated metrics (faithfulness — does the answer actually follow from the retrieved context — and relevance) and human review from subject-matter experts on each team, since automated scoring alone tends to miss domain-specific errors.

Citation accuracy checks. Verify that cited sources actually contain the claim attributed to them. This can be partly automated (checking that cited text overlaps with the source chunk) but benefits from spot-checks by a person.

Access control testing. Run adversarial tests where users from one team try to retrieve another team's restricted content, directly or through indirect prompting, and confirm the access layer blocks it every time.

Ongoing monitoring. Log every query and answer in production, sample a portion weekly for review, and track metrics over time (accuracy, retrieval hit rate, user thumbs-up/down feedback) so degradation gets caught early — especially as new documents get added and the index grows.

User feedback loop. A simple thumbs up/down with an optional comment on each answer gives a continuous, low-effort accuracy signal and helps flag which document types or teams need better ingestion or chunking.

